Open Authorization: Protocol Design and Formal Analysis with OFMC

Kristófer Birgir Hjörleifsson - s253155 Mikael Máni Eyfeld Clarke - s253024

Hand-in: Mar 16, 2026

Abstract

We design and incrementally refine an open authorization protocol in which a user A delegates read-only access to a photo resource at server B to a photo book service P, using an honest identity provider IdP. The protocol is specified in AnB and analyzed using OFMC under the Dolev–Yao attacker model. This report documents the development process, discovered attacks, design iterations, and the final protocol, including weekly special tasks.

Contents

1 Group Setup and Section Ownership **3**

2 Resources Used **3**

3 Scenario and Security Goals **3**
3.1 Scenario . . . 3
3.2 Agents . . . 3
3.3 Security and Functional Goals . . . 4

4 Week 2: Informal Design and Static Dolev–Yao Analysis **4**
4.1 Informal Outline . . . 4
4.2 First AnB Model (Simplified) . . . 4
4.3 OFMC Result . . . 4
4.4 Special Task: Static Analysis (Passive Intruder) . . . 4

5 Week 3: Identity Provider and Lazy Intruder Executability **5**
5.1 Protocol Refinement . . . 5
5.2 Special Task: Lazy Intruder Method (P Executable by Intruder) . . . 5

6 Week 4: Typing, Formats, and Public-Key Lookup **5**
6.1 Type-Flaw Resistance via Formats . . . 6
6.2 Public-Key Lookup Subprotocol . . . 6
6.3 Special Task: Type-Flaw Resistance Proof . . . 6

7 Week 5: Channels (TLS-like Pseudonymous Channels) **6**
7.1 Channel Notation . . . 6
7.2 Channel Variant . . . 7
7.3 Special Task: What Was Replaced . . . 7

8 Week 6: Privacy and Guessable Password **7**
8.1 Guessable Secrets . . . 7
8.2 Security with Guessable Password . . . 7
8.3 Special Task: Insecure Variant Demonstration . . . 7

9 Final Protocol (AnB Specification Summary) **8**
9.1 Final Message Flow 8
9.2 Initial Knowledge 8
9.3 Goals 8
9.4 Verification Results 8

10 Assumptions, Simplifications, and Limitations **9**
10.1 Assumptions 9
10.2 Simplifications 9
10.3 Limitations 9

11 Conclusion **9**

A Submission Map (Where to Find Things) **10**

1 Group Setup and Section Ownership

Section author: Kristófer Birgir Hjörleifsson

- **Group members:** Kristófer Birgir Hjörleifsson, Mikael Máni Eyfeld Clarke
- **Repository:** (https://github.com/kristoferbirgir/Project1-LogicForSecurity)
- **Submitted artifacts:** This report (PDF), AnB files, and the lab logbook.

Section ownership requirement (DTU): Each section in this report lists exactly one author. Any section without a single designated author may be counted as not submitted.

2 Resources Used

Section author: Kristófer Birgir Hjörleifsson

- Course slides and lecture notes (weeks 2–7).
- OFMC / AnB documentation and examples from the course material.
- AVISPA project documentation for AnB syntax reference.
- AI tools: GitHub Copilot used for protocol development iteration and documentation.
- Group discussions between team members.

3 Scenario and Security Goals

Section author: Mikael Máni Eyfeld Clarke

3.1 Scenario

A stores holiday photos at server B. A wants to create a photo book using service P and wishes to authorize P to access a subset of A's photos at B without downloading and re-uploading them. A authenticates using an identity provider IdP via a shared password. The password is considered guessable and must not be used as an encryption key. All parties except A have public/private keys. All parties know IdP's public key, and IdP knows all public keys. IdP is assumed honest and trusted.

3.2 Agents

- **A (User):** Owns photos, authenticates via password, no cryptographic keys.
- **B (Photo Server):** Stores A's photos, has key pair `pk(B)/inv(pk(B))`.
- **P (Printing Service):** Wants to access A's photos, has key pair `pk(P)/inv(pk(P))`.
- **IdP (Identity Provider):** Authenticates A, issues authorization tokens, has key pair `pk(IdP)/inv(pk(IdP))`.

3.3 Security and Functional Goals

We target the following goals:

- **Authorization:** B should only release photos if authorized by A via IdP.
- **Password secrecy:** A's password must not be revealed to P, B, or the intruder.
- **Photo confidentiality:** Photos should remain secret between authorized parties.
- **Authentication:** B must be able to verify IdP signed the authorization token.

4 Week 2: Informal Design and Static Dolev–Yao Analysis

Section author: Kristófer Birgir Hjörleifsson

4.1 Informal Outline

The basic protocol flow:

1. A requests authorization from IdP (encrypted with IdP's public key)
2. IdP creates signed token authorizing P to access A's photos at B
3. A forwards token to P
4. P presents token to B (encrypted with B's public key)
5. B verifies token and sends photos to P

4.2 First AnB Model (Simplified)

Initial simplification: A has a key pair (temporary, removed in Week 3).

Listing 1: Week 2 protocol excerpt

```
Actions:
  A -> IdP: {A, P, B}(pk(IdP))
  IdP -> A: {A, P, B}(inv(pk(IdP)))
  A -> P: {A, P, B}(inv(pk(IdP)))
  P -> B: {{A, P, B}(inv(pk(IdP)))}(pk(B))
  B -> P: {photos(A)}(pk(P))
```

4.3 OFMC Result

OFMC found a **signature forgery attack**: the intruder creates a token signed with their own key, and B cannot distinguish `pk(IdP)` from `pk(i)`.

4.4 Special Task: Static Analysis (Passive Intruder)

Assumption: Intruder only eavesdrops (passive), does not inject messages.

What intruder observes:

- Message 1: `{A, P, B}(pk(IdP))` – cannot decrypt (no `inv(pk(IdP))`)
- Message 2: `{A, P, B}(inv(pk(IdP)))` – can verify signature, learns A, P, B
- Message 3: Same signed token – learns nothing new

- Message 4: `{{token}}(pk(B))` – cannot decrypt outer layer
- Message 5: `{photos(A)}(pk(P))` – cannot decrypt

Intruder's knowledge (passive): Agent identities A, P, B, IdP (from signed token). **Cannot learn:** photos, password, or private keys.

Conclusion: Protocol is secure against passive eavesdropping. The signature forgery attack requires active message injection.

5 Week 3: Identity Provider and Lazy Intruder Executability

Section author: Mikael Máni Eyfeld Clarke

5.1 Protocol Refinement

Key change: A no longer has a key pair. A authenticates to IdP using password.

Listing 2: Week 3: Password authentication

```
A -> IdP: {A, P, B, pwd(A)}(pk(IdP))
```

The password is encrypted under IdP's public key, so the intruder cannot learn it. Password is NOT used as an encryption key – only as a credential.

5.2 Special Task: Lazy Intruder Method (P Executable by Intruder)

We show role P is executable by constructing all outgoing messages from intruder's knowledge.

Role P receives and sends:

1. Receives: `token = {IdP, A, P, B}(inv(pk(IdP)))`
2. Sends: `{P, token}(pk(B))`

Lazy Intruder Analysis:

- **Step 1:** Intruder intercepts message 3 (A → P), learns `token`
- **Step 2:** Intruder knows `P` (public agent name)
- **Step 3:** Intruder knows `pk(B)` (public key, in initial knowledge)
- **Step 4:** Intruder computes `{P, token}(pk(B))` using encryption rule

Conclusion: Role P is executable by the intruder. Every message P sends can be constructed from publicly available information and intercepted messages. This is expected – P acts as a relay.

6 Week 4: Typing, Formats, and Public-Key Lookup

Section author: Kristófer Birgir Hjörleifsson

6.1 Type-Flaw Resistance via Formats

Type flaws occur when an intruder substitutes a message of one type where another is expected. We prevent this by:

- Including agent identities in messages as implicit format tags
- Using different tuple structures for different message types

Listing 3: Message formats

```
Msg 1: {A, P, B, pwd(A)}(pk(IdP)) % 4-tuple with password
Msg 2: {IdP, A, P, B}(inv(pk(IdP))) % 4-tuple, agents only
Msg 4: {P, {token}(inv(pk(IdP)))}(pk(B)) % Nested structure
Msg 5: {B, photos(A)}(pk(P)) % 2-tuple
```

6.2 Public-Key Lookup Subprotocol

A separate protocol allows A to obtain public keys from IdP:

Listing 4: Key lookup protocol

```
A -> IdP: {A, B, pwd(A)}(pk(IdP))
IdP -> A: {B, pk(B)}(inv(pk(IdP)))
```

6.3 Special Task: Type-Flaw Resistance Proof

Definition: Protocol is type-flaw resistant if for any two non-variable messages M_1, M_2 in SMP, if they have a unifier σ, then $\text{type}(M_1) = \text{type}(M_2)$.

Analysis of message types:

Message	Structure	Type
Msg 1	(Agent, Agent, Agent, Function)	AuthRequest
Msg 2	(Agent, Agent, Agent, Agent)	Token
Msg 4 outer	(Agent, SignedMsg)	PhotoRequest
Msg 5	(Agent, Function)	PhotoResponse

Unification check:

- Msg 1 vs Msg 2: Different types (`pwd(A)` $\neq$ Agent) – no unifier
- Msg 4 vs Msg 5: Different types (SignedMsg $\neq$ Function) – no unifier

Conclusion: Protocol is type-flaw resistant.

7 Week 5: Channels (TLS-like Pseudonymous Channels)

Section author: Mikael Máni Eyfeld Clarke

7.1 Channel Notation

Notation	Meaning
`A -> B: M`	Insecure (Dolev-Yao)
`A *-> B: M`	Confidential only (TLS, client not auth)
`A ->* B: M`	Authenticated only (signed)
`A *->* B: M`	Secure channel (mutual TLS)

7.2 Channel Variant

Listing 5: Week 5: TLS channels

```
A *->* IdP: A, P, B, pwd(A) % TLS to IdP
IdP *->* A: {IdP, A, P, B}(inv(pk(IdP))) % Signed token
A *-> P: {token}(inv(pk(IdP))) % Forward token
P *->* B: P, {token}(inv(pk(IdP))) % Present to B
B *->* P: B, photos(A) % Response
```

7.3 Special Task: What Was Replaced

Original	Channel Version	Replaced?
`{M}(pk(IdP))`	`A *-> IdP: M`	Yes
`{M}(pk(B))`	`P *-> B: M`	Yes
`{M}(pk(P))`	`B *-> P: M`	Yes
`{M}(inv(pk(IdP)))`	Cannot replace	**No**

Why signature cannot be replaced: B needs to verify IdP approved the authorization, but B has no direct TLS channel to IdP. The token travels through A and P, so B must verify the signature cryptographically.

8 Week 6: Privacy and Guessable Password

Section author: Kristófer Birgir Hjörleifsson

8.1 Guessable Secrets

A guessable secret (like a password) can be subject to offline dictionary attacks. The intruder captures encrypted data containing the password and tries decrypting with guessed passwords.

8.2 Security with Guessable Password

Our protocol protects against guessing attacks because:

- Password is sent over secure channel (`A *->* IdP`)
- Intruder never sees ciphertext containing password
- No offline verification possible

OFMC Goal: `pwd(A) guessable secret between A, IdP`
Result: No guessing attack found (only signature forgery under dishonest IdP).

8.3 Special Task: Insecure Variant Demonstration

We created an insecure variant to demonstrate the attack:

Listing 6: Insecure version (vulnerable to guessing)

```
A -> IdP: {A, P, B, pwd(A)}(pk(IdP)) % Regular encryption
```

OFMC Result: `ATTACK_FOUND (guesswhat)`

Listing 7: Guessing attack trace

```
(A,1) -> i: {A,P,B,pwd(A)}_(pk(IdP))
i can produce secret {A,P,B,guessPW}_(pk(IdP))
```

Conclusion: Encryption alone does not protect guessable secrets. Secure channels (TLS) prevent offline guessing by hiding the ciphertext entirely.

9 Final Protocol (AnB Specification Summary)

Section author: Mikael Máni Eyfeld Clarke

9.1 Final Message Flow

Listing 8: Final protocol (photo_auth_final.AnB)

```
A *->* IdP: A, P, B, pwd(A)
IdP *->* A: {IdP, A, P, B}(inv(pk(IdP)))
A *-> P: {IdP, A, P, B}(inv(pk(IdP)))
P *->* B: P, {IdP, A, P, B}(inv(pk(IdP)))
B *->* P: B, photos(A)
```

Step-by-step:

1. A authenticates to IdP with password over TLS
2. IdP creates signed authorization token
3. A forwards token to P over confidential channel
4. P presents token to B with mutual TLS
5. B verifies token signature and sends photos to P

9.2 Initial Knowledge

Agent	Knowledge
A	A, B, P, IdP, pwd(A)
B	A, B, P, IdP, pk(B), pk(IdP), inv(pk(B)), photos(A)
P	A, B, P, IdP, pk(B), pk(P), pk(IdP), inv(pk(P))
IdP	A, B, P, IdP, pk(B), pk(P), pk(IdP), inv(pk(IdP)), pwd(A)

9.3 Goals

- `B authenticates IdP on IdP, A, P, B` – B verifies IdP signed token
- `photos(A) secret between B, P` – Photos only known to B and P
- `pwd(A) guessable secret between A, IdP` – Password protected

9.4 Verification Results

OFMC Result: ATTACK_FOUND (signature forgery)

The attack involves the intruder acting as IdP, signing a forged token with their own key. This attack is **excluded by assumption** since IdP is assumed honest.

Under honest IdP assumption: Protocol is secure.

10 Assumptions, Simplifications, and Limitations

Section author: Kristófer Birgir Hjörleifsson

10.1 Assumptions

- **IdP is honest:** The identity provider never acts maliciously or is compromised. Attacks involving dishonest IdP are excluded.
- **Secure channels abstract TLS:** We assume TLS provides confidentiality and authentication as modeled.
- **Keys are pre-distributed:** Servers know each other's public keys.

10.2 Simplifications

- **Photos as abstract secret:** Modeled as `photos(A)` rather than actual data.
- **No scope/folder:** Token authorizes access to all photos, not a specific subset.
- **No timestamps/nonces:** Token has no expiration or freshness mechanism.
- **Single session:** OFMC verifies bounded number of sessions.

10.3 Limitations

- **Signature forgery persists:** B cannot distinguish pk(IdP) from pk(i) without out-of-band trust establishment.
- **No revocation:** Once issued, tokens cannot be revoked.
- **IdP single point of failure:** If IdP is compromised, all security fails.

11 Conclusion

Section author: Mikael Máni Eyfeld Clarke

We designed an open authorization protocol allowing user A to authorize printing service P to access photos at server B, using identity provider IdP for authentication.

Main attack encountered: Signature forgery – the intruder creates tokens signed with their own key. This is a fundamental Dolev-Yao limitation where B cannot distinguish honest from dishonest signers without pre-established trust.

Solution: We assume IdP is honest (as per assignment). Under this assumption, the protocol provides:

- Authentication of IdP's authorization via signed tokens
- Confidentiality of photos (only B and P)
- Protection of guessable password via secure channels

Final protocol uses: TLS channels for confidentiality, cryptographic signatures for authorization tokens that travel through untrusted intermediaries.

A Submission Map (Where to Find Things)

Section author: Kristófer Birgir Hjörleifsson

- Lab logbook: `logbook/logbook.md`
- AnB protocol versions:
  - `anb/week2_v1.AnB` – Basic protocol
  - `anb/week3_v1.AnB` – Password authentication
  - `anb/week4_v1.AnB` – Type-flaw resistant
  - `anb/week5_v1.AnB` – TLS channels
  - `anb/week5_v1_tls.AnB` – Server-side only TLS
  - `anb/week6_v1.AnB` – Guessable password
  - `anb/week6_insecure.AnB` – Insecure variant (demo)
- Final AnB files:
  - `anb/final/photo_auth_final.AnB` – Final protocol
  - `anb/final/key_lookup.AnB` – Key lookup subprotocol